

Bản trình chiếu: Tinh luyện và nén cấu trúc dữ liệu

Cao Qinxiang

2008

Nguồn gốc: Tinh luyện và nén cấu trúc dữ liệu: bản trình chiếu

IOI China candidate team papers / 2008 / Day 1 / Cao Qinxiang / slides.ppt

1 Mạch chính của slide

Bản trình chiếu đi cùng bài viết *Tinh luyện và nén cấu trúc dữ liệu*. Luồng văn bản trong tệp PowerPoint cũ chỉ trích xuất được một phần, nhưng các tiêu đề và ví dụ chính khớp với bài Word: *Ural 1568 Train Car Sorting*, *CEOI 2007 Necklaces*, *Hide and Seek*, thống kê trên cây, truy vấn trên đường trong trie, và hai ví dụ dùng cây DFS/BFS. Bản dịch này giữ lại mạch trình bày của slide, không lặp toàn bộ chứng minh dài của bài viết.

Thông điệp trung tâm: một cấu trúc dữ liệu tốt không nhất thiết lưu nhiều thông tin hơn. Trái lại, ta thường thắng bằng cách xác định thông tin nào là vô dụng, thông tin nào bị lặp lại, và cấu trúc nào có thể ép về một mô hình đơn giản hơn.

2 Tinh luyện: bỏ thông tin rỗng

Ví dụ mở đầu là *Train Car Sorting*. Bài toán có thể mô tả một hoán vị bằng “ma trận mẹ”. Nếu lưu cả ma trận thì chi phí trở thành bậc hai, nhưng trong ma trận chỉ có n ô khác rỗng. Các ô bằng không không ảnh hưởng tới thao tác đọc hàng/cột và không cần nằm trong cấu trúc dữ liệu.

Vì vậy slide nhấn mạnh bước tinh luyện:

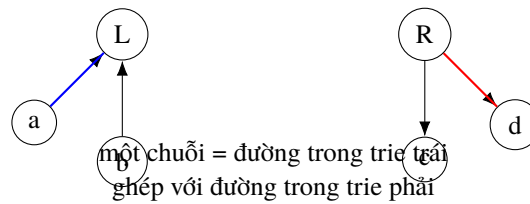
- tìm mô hình thuận tiện để chứng minh thuật toán;
- nhìn lại mô hình đó và xóa các phần không tham gia vào tính toán;
- lưu các đối tượng thật sự hoạt động, không lưu phần nền trống.

Với *Train Car Sorting*, cách lưu này giảm bộ nhớ xuống $\mathcal{O}(n)$ và tổng thời gian tỷ lệ với kích thước lời giải xuất ra.

3 Thu gọn: chia sẻ phần chung

CEOI 2007 Necklaces là ví dụ chính của thu gọn thông tin lặp lại. Mỗi thao tác tạo một chuỗi mới từ chuỗi cũ bằng cách thêm hoặc xóa ở đầu trái hay đầu phải. Nếu sao chép toàn bộ chuỗi sau mỗi thao tác, bộ nhớ có thể tăng tới $\mathcal{O}(n^2)$. Nhưng hai phiên bản liên tiếp chỉ khác một phần tử, nên phần chung cần được chia sẻ.

Bài viết dùng cấu trúc left-right tree: một chuỗi được tách thành phần trái và phần phải. Phần trái được biểu diễn bằng một đường trong trie trái, phần phải bằng một đường trong trie phải. Một phiên bản chuỗi chỉ cần bốn con trỏ tới hai đường này.



Hình 1: Sơ đồ hóa left-right tree: nhiều phiên bản chuỗi chia sẻ các đường trie chung.

Vấn đề còn lại là đi nhanh trên một đường trie, chẳng hạn tìm con kế tiếp của một tổ tiên trên đường tới lá. Slide đưa về bảng nhảy tổ tiên giống LCA: lưu độ sâu và các “siêu cha”, rồi nhảy theo lũy thừa hai.

4 Nén cây thành dãy

Hide and Seek minh họa việc nén cấu trúc cây thành chuỗi ngoặc DFS. Khoảng cách giữa hai đỉnh có thể đọc từ đoạn ngoặc giữa hai lần xuất hiện: sau khi triệt tiêu các cặp ngoặc khớp, số dấu đóng còn lại là số bước đi lên, số dấu mở còn lại là số bước đi xuống.

Nhờ đó mỗi đoạn được mô tả bởi một cặp (a, b) , và phép ghép hai đoạn trở thành một phép hợp nhất cục bộ. Cây đoạn trên chuỗi ngoặc lưu thêm các đại lượng tốt nhất ở tiền tố/hậu tố, từ đó trả lời đường kính của các đỉnh đang được đánh dấu và hỗ trợ đảo màu một đỉnh.

5 Hai thứ tự DFS

Trong bài thống kê trên cây, với mỗi đỉnh v cần đếm số hậu duệ có nhãn nhỏ hơn v . Thay vì duyệt lại toàn bộ cây con của từng đỉnh, slide dùng hai thứ tự:

- DFS thông thường;
- DFS sau khi đảo thứ tự con tại mỗi nút.

Giao của hai vùng “sau v ” trong hai dãy này chính là phần hậu duệ cần đếm, trừ các hiệu chỉnh tổ tiên. Fenwick tree xử lý số lượng nhỏ hơn trên dãy và trên đường gốc-đỉnh, đưa bài toán về $\mathcal{O}(n \log n)$.

6 DFS và BFS trong đồ thị

Phần cuối của slide chuyển từ cây sang đồ thị. Với cây DFS của đồ thị vô hướng, không có cạnh ngang; mọi cạnh ngoài cây nối một đỉnh với tổ tiên của nó. Tính chất này làm các bài liên quan tới cầu, khớp, hướng cạnh và điều kiện liên thông trở nên dễ kiểm soát hơn.

Ngược lại, cây BFS giữ thông tin khoảng cách theo lớp. Trong bài chọn cây khung có đường kính nhỏ, thử từng gốc BFS và xét độ sâu các lớp cho ta ứng viên tốt; khi đường kính tối ưu lẻ, tâm có thể là một cạnh nên cần kiểm tra thêm điều kiện hai phía của cạnh tâm.

7 Kết luận của slide

Ba động tác “tinh luyện, nén, thu gọn” thực chất đều nhằm giảm quy mô thông tin phải xử lý. Sau khi ép cấu trúc phức tạp về dãy, trie dùng chung, chuỗi ngoặc, cây DFS hoặc cây BFS, ta có thể dùng những công cụ quen thuộc như cây đoạn, Fenwick tree và bảng nhảy tổ tiên. Đó là cách biến cấu trúc dữ liệu khó thành cấu trúc nhỏ hơn, nhanh hơn và dễ cài đặt hơn.